

cally injected via the Laravel service container. Before we go any further, a quick note on what a service container is. It is a rather essential tool to help manage class dependencies that are injected via the constructor method. Sometimes the setter method is used.

Data Storage in Laravel's Sessions

No prizes for guessing the need to explain the quintessential feature of sessions-Data storage and how it is done. Data storage is essentially done by the `put()` method. The `put()` method protects your application from any possible cross site forgeries.

The code below can be used via request instance

```
$request->session()->put('key', 'value');
```

The code below can be used via global helper

```
session(['key' => 'value']);
```

Deleting Data in Laravel's Sessions

There are two methods to delete or remove data that is stored in Laravel's sessions. One type is to remove only parts or some items of the data and not the data in all its entirety. This is the `forget` method. This can be done by `$request->session()->forget('key');`

To remove all the stored data a different method is used. This is the `flush` method. This technique will first use the `pull()` method to retrieve data from session following which the data will be deleted after. Code for flush method is `$request->session()->flush();`

Sending Emails

The aim of Laravel was to simplify various and wide range functions of web development.

Some of the common tasks which are quintessential to a website have been made available to achieve with much ease. One of these features is sending emails. This has been reviewed with a gold star amongst web developer peers. The mail function called 'Swift Mailer' is built for this. The swiftmailer uses the library function to be able to send emails without causing much of a hassle. The blade syntax is used to inject data into the templates. Emails use the blade template to create email contents.